3D-Printer-Simulation

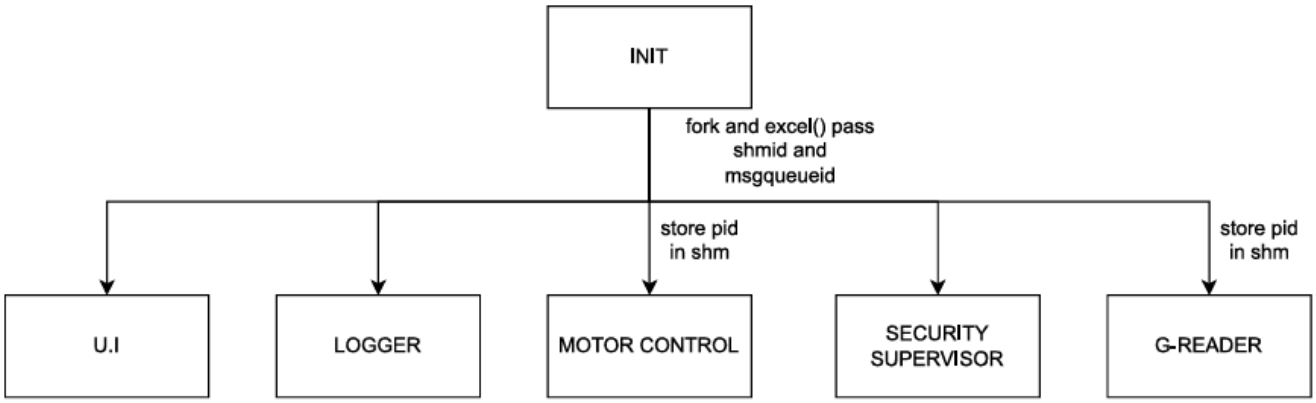
Scenario Description

A real 3D printer consists of multiple components working simultaneously:

- A controller reads G-code instructions
- Motors execute movement commands
- Temperature is continuously monitored
- A supervisor ensures safe operation
- A UI displays real-time printer state

This project simulates these components using independent processes and threads that communicate using IPC mechanisms.

Process Level Design :



Parent: INIT

Children: G_READER , MOTOR_CONTROL , LOGGER , U.I ,SUPERVISOR.

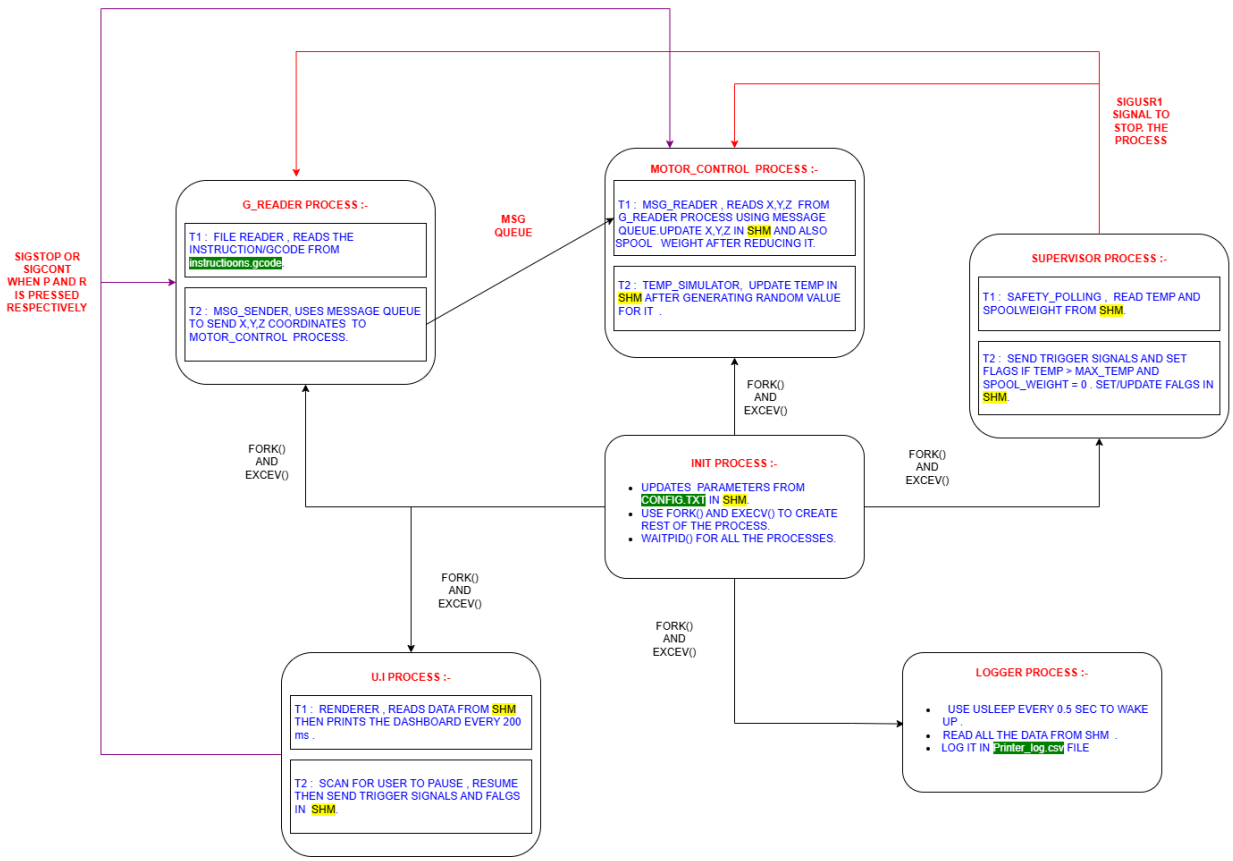
Created using fork() and excel().

excel() passes shm_id and msg_id.

Also INIT Stores pids of motor_control and g_reader process in shm so they can use it for message queue.

Parent monitors them with waitpid().

Thread Level Design :



1. Initialization Process (init.c)

Responsibilities:

- Creates shared memory and message queue
- Initializes global printer state
- Reads configuration parameters
- Spawns all system processes using fork() and execl()
- Stores child PIDs in shared memory
- Waits for execution and performs cleanup

2. G-code Reader Process (g_reader.c)

- Reads G-code instructions from file
- Implements producer-consumer model using threads:

Threads:

- File Reader Thread : Reads Gcode from instructions.gcode or whichever file given in config.txt
- Message Sender Thread : Uses msg queue to send this code to motor_control process.
- Uses:
    - Mutex
    - Condition variables
- Sends commands to motor control via message queue

- Sends "DONE" when file processing completes

3. Motor Control Process (motor_control.c)

Handles core printer simulation.

Thread 1: Message Reader

- Receives commands from message queue
- Parses G-code (G0/G1)
- Updates X, Y, Z position in shared memory
- Simulates filament consumption

Thread 2: Temperature Simulator

- Updates temperature every second
- Increases temperature during printing
- Cools when idle
- Triggers emergency stop if temperature exceeds limit

4. Supervisor Process (supervisor.c)

Ensures system safety.

Thread 1: Safety Polling

- Monitors temperature and spool weight
- Stops system if limits are exceeded

Thread 2: Trigger Monitor

- Monitors shared memory flags:
    - emergency_stop
    - job_done
- Sends stop signals to worker processes

5. UI Process (ui.c)

Provides real-time terminal visualization.

Features:

- Displays two views:
    - Top View (X-Y plane)
    - Front View (X-Z plane)
- Shows:
    - Print head position
    - Temperature
    - Spool weight

- System status

Threads:

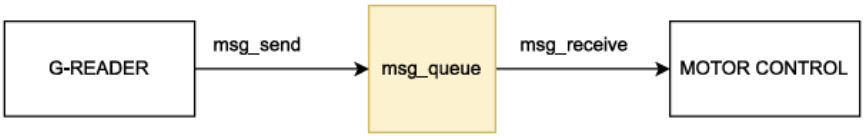
- Renderer Thread:
  - Updates display continuously
  - Tracks filament placement
- Input Thread:
  - Handles user input in raw mode

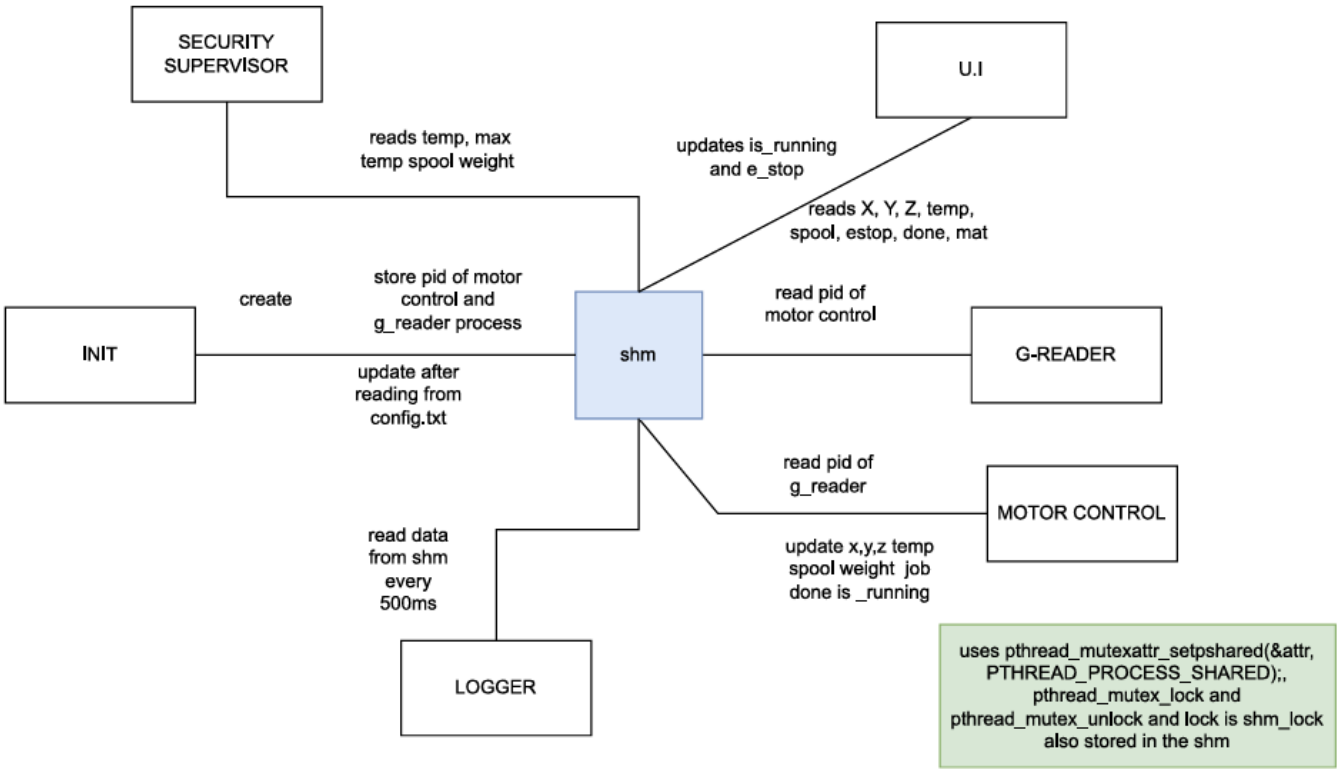
Controls:

- P → Pause printing (SIGSTOP)
- R → Resume printing (SIGCONT)
- Q → Quit / Emergency stop (SIGUSR1)

Uses termios for non-blocking input.

Inter-Process Communication (IPC) :





1. Message Queue

Used for command transfer:

G-code Reader → Motor Control

Functions:

- msgget()
- msgsnd()
- msgrcv()

2. Shared Memory

Used for global system state:

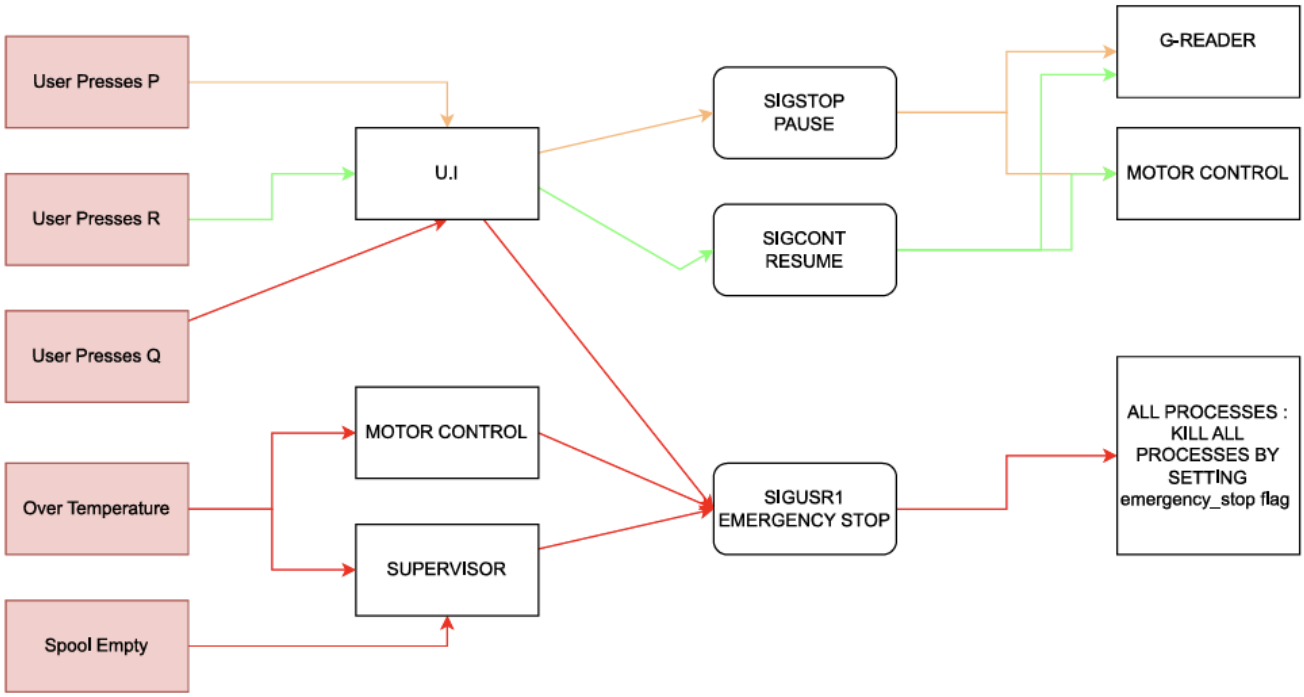
- Position (X, Y, Z)
- Temperature
- Spool weight
- Flags:
    - is_running
    - emergency_stop
    - job_done

Functions:

- shmget()
- shmat()
- shmdt()

A mutex inside shared memory ensures safe concurrent access.

Signal Handling :



Signals are used for system control.

SIGSTOP

Pauses printing (used by UI)

SIGCONT

Resumes printing (used by UI)

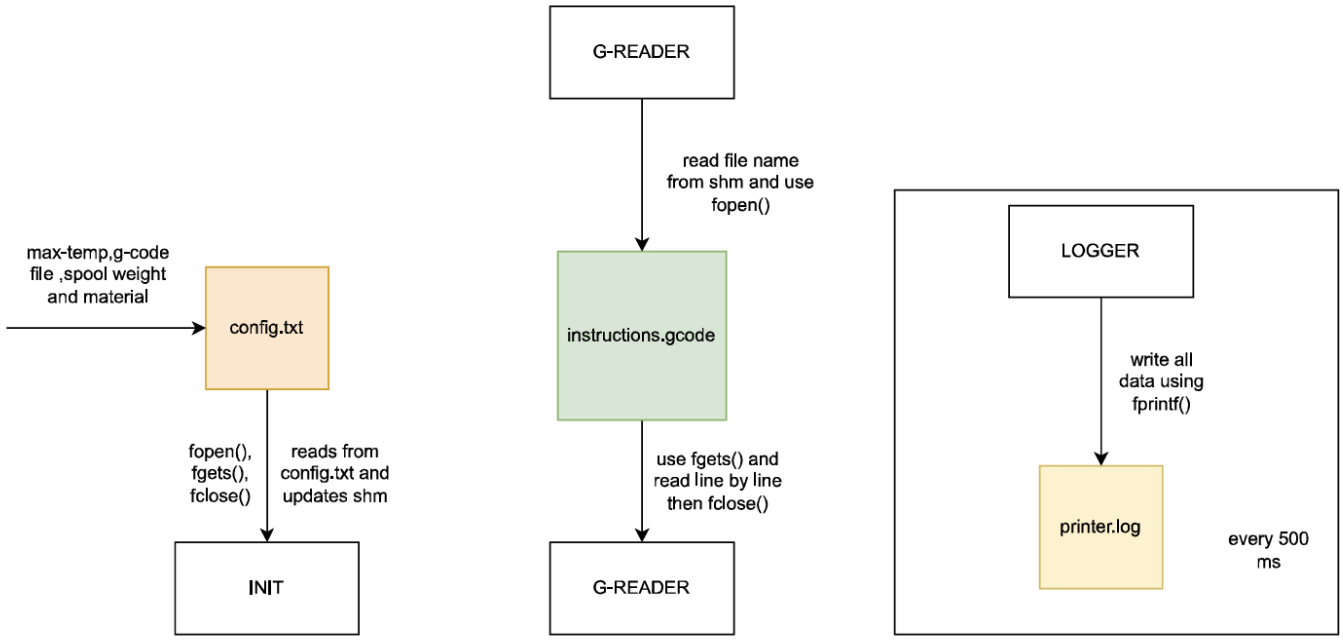
SIGUSR1

Triggers emergency stop

Used by:

- UI (P,Q,R pressed by user)
- Supervisor (overtemperature,empty spool)
- Motor control (overtemperature)

File Handling



1. Configuration Architecture (The INIT Process)

Before the printing starts, a dedicated Initialization (INIT) routine handles the system setup.

- Manual Configuration: The user manually updates config.txt with hardware limits like max-temp, material_type, and the g-code_filename.
- Parsing Routine: The INIT process uses fopen(), fgets(), and sscanf() to parse these key-value pairs.
- State Update: Once parsed, INIT updates the Shared Memory (SHM) so all other processes (like the Motor Controller) know the safety thresholds.

2. G-Code Stream (The G-READER Process)

The G-READER process acts as the that feeds the movement pipeline.

- Dynamic Loading: It reads the target filename directly from Shared Memory (provided by INIT) and opens it using fopen().
- Line-by-Line Execution: It uses fgets() to read the file one instruction at a time.

3. System Logging (The LOGGER Process)

To ensure high-fidelity monitoring, a separate LOGGER process runs concurrently.

- Frequency: Every 500 ms, the LOGGER wakes up to capture a snapshot of the current system state.
- Data Source: It pulls real-time coordinates (X, Y, Z) and temperature along with other things from Shared Memory.
- Persistence: It uses fprintf() to write this data into printer.log, creating a permanent record for debugging or print recovery.